

Irrecoverable Schedule

Understanding transaction dependencies, commit order, and recovery failure

 One-Line Definition

An irrecoverable schedule occurs when a transaction commits after reading dirty data from another transaction that subsequently fails and rolls back.

1. INTRODUCTION TO RECOVERABILITY

In concurrent database systems, transaction schedules are analyzed primarily from two perspectives: **Serializability** (correctness of execution) and **Recoverability** (ability to safely recover database state upon failure).

An **irrecoverable schedule** arises when a committed transaction relies on uncommitted data from an earlier transaction that eventually aborts. Because the reader has already finalized its changes (`COMMIT`), the database engine cannot roll back the reader without violating the Durability (D in ACID) property.

2. DETAILED STEP-BY-STEP SCHEDULE TRACE

Initial Database State: `A = 10` , `B = 20`

Time	Transaction 1 (T1)	Transaction 2 (T2)	State of A
t1	<code>READ(A)</code> → gets 10	—	10
t2	<code>A = A - 5</code> (local)	—	10
t3	<code>WRITE(A)</code> → local update 5	—	5 (dirty)
t4	—	<code>READ(A)</code> → gets 5 (dirty read)	5 (dirty)
t5	—	<code>A = A - 2</code> (local)	5 (dirty)
t6	—	<code>WRITE(A)</code> → 3	3
t7	—	<code>COMMIT</code> T2 Finalized	3 (committed)
t8	<code>READ(B)</code> → Error / Failure!	—	3
t9	<code>ROLLBACK</code> T1 Reverted	—	Invalid!

3. WHERE DOES THE FAILURE OCCUR?

- T2 reads `A = 5` produced by T1 and immediately **COMMITs**.
- Later, T1 encounters a system error and triggers a **ROLLBACK**.
- T1's write (`10 → 5`) is undone, resetting A to 10.
- Conflict:** T2 depended on `A = 5`, which never officially existed. But T2 cannot be rolled back because it has already committed.

 The Core Mechanism

The database enters an inconsistent state because a committed transaction (T2) is built on top of dirty, aborted data from an uncommitted transaction (T1).

4. THE KEY EXECUTION PATTERN

To identify an irrecoverable schedule in exam questions, check for the exact execution sequence of writer and reader operations:

Execution Sequence

1. T1: WRITE(A) (Dirty write)



2. T2: READ(A) (Dependent read)



3. T2: COMMIT (Reader commits)



4. T1: FAIL / ROLLBACK (Writer aborts)

Core Memory Trick

W → R → C → F

Write (T1)

Read (T2)

Commit (T2)

Fail (T1)

If **Writer Fails AFTER Reader Commits**, the schedule is **IRRECOVERABLE!**

5. REAL-LIFE ANALOGY

Financial Information Analogy

Person A informs **Person B**: "I have transferred ₹5,000 to your account."

Trusting this unverified statement, **Person B** signs a contract to purchase an item (COMMIT).

Later, **Person A's** bank transaction fails (ROLLBACK) and the ₹5,000 was never delivered. **Person B** cannot undo the signed contract. This non-recoverable dependency mirrors an irrecoverable schedule.

6. QUICK REVISION SUMMARY

Event Sequence	Operation Meaning	Impact on Recovery
T1 WRITE(A)	T1 modifies data item A	Uncommitted dirty write created
T2 READ(A)	T2 reads T1's modified value	Dependency established (T2 depends on T1)
T2 COMMIT	T2 permanently saves its state	T2 can no longer be rolled back
T1 FAIL & ROLLBACK	T1 fails and undoes modifications	IRRECOVERABLE — T2 used invalid data!

⚠ Most Important Exam Takeaway

A schedule is recoverable **ONLY IF** every transaction T2 that reads data written by T1 delays its COMMIT until **AFTER** T1 commits. If T2 commits before T1, and T1 subsequently fails, the schedule becomes **irrecoverable**.